

Functions in Python

Contents

1	Introduction	4
2	What is Function?	4
3	Why Functions are Important	5
4	Function Definition	5
4.1	Understanding Syntax	5
5	Function Call	5
6	Calling Function Multiple Times	6
7	Function Parameters	6
8	What is Parameter?	6
9	Arguments	6
10	Example	7
11	Default Parameters	7
12	Overriding Default Value	7
13	Positional Arguments	7
14	Wrong Position Example	8
15	Keyword Arguments	8
16	Important Rule	8
17	Return Statement	9
18	Calling Function with Return Value	9
19	return vs print	9
19.1	print()	9
19.2	return	9

20 Flexible Function Example	9
21 isinstance() Function	10
22 Variable Number of Arguments	10
23 Star Arguments	10
24 Understanding *numbers	11
25 Example	11
26 Similar to sum() Function	11
27 Double Star Arguments	11
28 Understanding **info	11
29 Single * vs Double **	12
29.1 Single *	12
29.2 Double **	12
30 Example Function Structure	12
31 Real Example: Grade Calculator	13
32 Calling Grade Function	13
33 Why Functions Matter Here	13
34 Using Functions in Menu Programs	14
35 Good Function Naming	14
36 Final Thoughts	14
37 Exercises	16
37.1 Exercise 1: Greeting Function	16
37.2 Exercise 2: Addition Function	16
37.3 Exercise 3: Even or Odd	16
37.4 Exercise 4: Largest Number	16
37.5 Exercise 5: Multiplication Table	16
37.6 Exercise 6: Student Grade	16
37.7 Exercise 7: Sum Using *args	16
37.8 Exercise 8: User Info	17
37.9 Exercise 9: Factorial	17
37.10Exercise 10: Prime Number	17

38 Challenge Questions	18
38.1 Challenge 1: Calculator App	18
38.2 Challenge 2: Login System	18
38.3 Challenge 3: Student Result System	18
38.4 Challenge 4: Shopping Bill	18
38.5 Challenge 5: Mini Bank System	19
38.6 Challenge 6: Voting System	19
38.7 Challenge 7: Password Strength Checker	19
38.8 Challenge 8: Menu-driven Student System	19
38.9 Challenge 9: Custom max() Function	19
38.10Challenge 10: Function-based Quiz Game	20
39 Homework Advice	20

1 Introduction

Till now we wrote programs like this:

```
print("Welcome")
print("Welcome")
print("Welcome")
print("Welcome")
```

Or copied same logic multiple times.

Example:

- Grade calculation
- Login checking
- Participant display

Repeated code creates:

- Bigger programs
- Messy programs
- Difficult maintenance

That is why:

Functions exist.

2 What is Function?

Function is:

Block of code that represents a specific task.

Another way to think:

Function is variable that stores group of code.

Examples:

- Send email
- Calculate marks
- Login user
- Print invoice

3 Why Functions are Important

Functions help us:

- Reuse code
- Reduce repetition
- Organize programs
- Make debugging easier

Professional software development is heavily function-based.

Without functions:

Programs become giant noodles of confusion.

4 Function Definition

```
def welcome_message():  
    print("Welcome, Ram")  
    print("Please verify your email")
```

4.1 Understanding Syntax

```
def
```

means:

Define function

```
welcome_message
```

is function name.

```
()
```

parentheses are required.

```
:
```

starts function block.

Indented lines belong to function.

5 Function Call

Defining function does not run it.

To run function:

```
welcome_message()
```

This is called:

Function Call

6 Calling Function Multiple Times

```
welcome_message()  
welcome_message()  
welcome_message()
```

Output repeats multiple times.
This is power of reusable code.
Write once. Use many times.

7 Function Parameters

Sometimes functions need data.

Example:

- User name
- Greeting message

```
def welcome_message(name, greeting):  
  
    print(greeting, name)  
  
    print("Please verify your email")
```

8 What is Parameter?

```
name  
greeting
```

are called:

Parameters

Parameters are variables inside function definition.
They wait for incoming data.

9 Arguments

```
welcome_message("Ram", "Good morning")
```

Here:

```
"Ram "  
"Good morning"
```

are called:

Arguments

Arguments are actual values sent into function.

10 Example

```
welcome_message("Ram", "Good morning")  
  
welcome_message("Hari", "Good afternoon")
```

Output:

```
Good morning Ram  
Please verify your email  
  
Good afternoon Hari  
Please verify your email
```

11 Default Parameters

Sometimes parameter should have default value.

```
def welcome_message(name, greeting="Welcome"):  
    print(greeting, name)
```

Now:

```
welcome_message("Ram")
```

Output:

```
Welcome Ram
```

Because:

```
greeting
```

used default value.

12 Overriding Default Value

```
welcome_message("Ram", "Good morning")
```

Output:

```
Good morning Ram
```

Provided argument replaces default value.

13 Positional Arguments

```
welcome_message("Ram", "Good morning")
```

Python matches values by position.

First value:

```
name
```

Second value:

```
greeting
```

14 Wrong Position Example

```
welcome_message("Good morning", "Ram")
```

Output becomes:

```
Ram Good morning
```

because positions got swapped.

Python does not magically read your mind.

Unfortunately.

15 Keyword Arguments

```
welcome_message(  
    greeting="Good morning",  
    name="Ram"  
)
```

This is called:

Keyword Argument

Now order does not matter.

Because we explicitly mention parameter names.

16 Important Rule

Non-default parameters must come first.

Wrong:

```
def welcome_message(greeting="Welcome", name):
```

Python gives:

```
SyntaxError
```

Correct:

```
def welcome_message(name, greeting="Welcome"):
```

17 Return Statement

Functions can send data back.

```
def add(a, b):  
    return a+b
```

18 Calling Function with Return Value

```
result = add(4, 5)  
  
print(result)
```

Output:

```
9
```

19 return vs print

Very important difference exists.

19.1 print()

Shows output on screen.

19.2 return

Sends value back to caller.

Example:

```
def add(a, b):  
    return a+b  
  
x = add(4, 5)
```

Now:

```
x
```

stores returned value.

20 Flexible Function Example

```
def add(a, b):  
  
    r = 0  
  
    if isinstance(a, list):
```

```
        for value in a:
            r += value

    else:
        r += a

    r += b

    return r
```

21 isinstance() Function

Checks datatype.

Example:

```
isinstance(5, int)
```

Output:

```
True
```

Another example:

```
isinstance([1,2], list)
```

Output:

```
True
```

22 Variable Number of Arguments

Sometimes we do not know how many values user sends.

Python provides:

```
*
```

23 Star Arguments

```
def add(*numbers, start=0):

    for value in numbers:
        start += value

    return start
```

24 Understanding *numbers

```
*numbers
```

means:

Accept any number of positional arguments.

Example:

```
add(1,2,3,4,5)
```

Python internally stores them as tuple.

25 Example

```
print(add(1,2,3,4,5, start=5))
```

Output:

```
20
```

26 Similar to sum() Function

Python already provides:

```
sum([1,2,3], start=5)
```

Our custom function behaves similarly.

27 Double Star Arguments

```
def show_info(**info):  
    print(info)
```

28 Understanding **info

```
**info
```

means:

Accept any number of keyword arguments.

Example:

```
show_info(  
    name="ram",  
    age=23,  
    address="ktm"  
)
```

Output:

```
{  
  'name': 'ram',  
  'age': 23,  
  'address': 'ktm'  
}
```

Python stores keyword arguments as dictionary.

29 Single * vs Double **

29.1 Single *

```
*args
```

Accepts:

- Positional arguments

Stored as:

- Tuple

29.2 Double **

```
**kwargs
```

Accepts:

- Keyword arguments

Stored as:

- Dictionary

30 Example Function Structure

```
def some_func(p1, p2=0, **kwargs):  
  
    if True:  
        return 0
```

Functions may contain:

- Loops
- Conditions
- Returns
- Complex logic

Functions are mini-programs inside bigger programs.

31 Real Example: Grade Calculator

Without function:

```
# repeated logic again and again
```

With function:

```
def calculate_grade(sub_marks):

    if sub_marks > 90:
        print("A+")

    elif sub_marks > 80:
        print("A")
```

32 Calling Grade Function

```
eng_marks = int(input("Enter English marks: "))

calculate_grade(eng_marks)
```

Again:

```
science_marks = int(input("Enter Science marks: "))

calculate_grade(science_marks)
```

33 Why Functions Matter Here

Imagine grade logic changes later.

Without function:

- Update logic everywhere

With function:

- Update only once

This is called:

Code Reusability

34 Using Functions in Menu Programs

Earlier we wrote giant menu programs.

Now:

```
if user_choice == "1":
    add_new_participant()

elif user_choice == "2":
    remove_participant()

elif user_choice == "3":
    show_participants()
```

Each task gets separate function.

Program becomes:

- Cleaner
- Easier to debug
- Easier to understand

35 Good Function Naming

Good names:

```
calculate_grade()
send_email()
show_participants()
```

Bad names:

```
abc()
x()
doit()
```

Function names should explain purpose clearly.

36 Final Thoughts

Today you learned:

- Function definition
- Function call
- Parameters
- Arguments
- Default parameters

- Keyword arguments
- return statement
- *args
- **kwargs
- Reusable code

Functions are one of the most important concepts in programming.
Once you master functions:

You stop writing scripts, and start building real software.

37 Exercises

37.1 Exercise 1: Greeting Function

Create function:

```
greet(name)
```

Print:

```
Hello <name>
```

37.2 Exercise 2: Addition Function

Create function that adds two numbers and returns result.

37.3 Exercise 3: Even or Odd

Create function:

```
is_even(number)
```

Return:

- True if even
- False if odd

37.4 Exercise 4: Largest Number

Create function that returns largest among two numbers.

37.5 Exercise 5: Multiplication Table

Create function that prints multiplication table of given number.

37.6 Exercise 6: Student Grade

Create function:

```
calculate_grade(marks)
```

Return grade.

37.7 Exercise 7: Sum Using *args

Create custom sum function using:

```
*args
```

37.8 Exercise 8: User Info

Create function using:

```
**kwargs
```

Print all user information.

37.9 Exercise 9: Factorial

Create factorial function.

Example:

```
factorial(5)
```

Output:

```
120
```

37.10 Exercise 10: Prime Number

Create function checking whether number is prime.

38 Challenge Questions

38.1 Challenge 1: Calculator App

Create calculator using functions:

- add()
- subtract()
- multiply()
- divide()

Use menu system.

38.2 Challenge 2: Login System

Create login function validating:

- Username
- Password

Allow only 3 attempts.

38.3 Challenge 3: Student Result System

Create functions for:

- Taking marks
- Calculating total
- Calculating percentage
- Showing grade

38.4 Challenge 4: Shopping Bill

Create billing system using functions.

Functions should:

- Add product
- Calculate total
- Apply discount
- Print final bill

38.5 Challenge 5: Mini Bank System

Functions:

- `deposit()`
- `withdraw()`
- `checkbalance()`

38.6 Challenge 6: Voting System

Create voting system using reusable functions.

38.7 Challenge 7: Password Strength Checker

Create function checking:

- Length
- Uppercase
- Number
- Special character

38.8 Challenge 8: Menu-driven Student System

Functions:

- `addstudent()`
- `removestudent()`
- `searchstudent()`
- `showstudents()`

38.9 Challenge 9: Custom `max()` Function

Create your own:

```
my_max()
```

without using built-in:

```
max()
```

38.10 Challenge 10: Function-based Quiz Game

Create quiz game using functions:

- `askquestion()`
- `calculatescore()`
- `showresult()`

39 Homework Advice

Functions become easy only after writing lots of programs.

At first:

“Why make function for everything?”

Later:

“How did I ever live without functions?”

That transformation is part of becoming programmer.